

MI DSP API

Version 3.2

© 2021 SigmaStar Technology Corp. All rights reserved.

SigmaStar Technology makes no representations or warranties including, for example but not limited to, warranties of merchantability, fitness for a particular purpose, non-infringement of any intellectual property right or the accuracy or completeness of this document, and reserves the right to make changes without further notice to any products herein to improve reliability, function or design. No responsibility is assumed by SigmaStar Technology arising out of the application or use of any product or circuit described herein; neither does it convey any license under its patent rights, nor the rights of others.

SigmaStar is a trademark of SigmaStar Technology Corp. Other trademarks or names herein are only for identification purposes only and owned by their respective owners.

REVISION HISTORY

Revision No.	Description	Date
3.0	Initial release	03/18/2021
	Added procfs introduction	08/25/2021
3.1	Modify the MI_DSP_FW_PACKING_TYPE_e enumeration variable	12/02/2021
3.2	Add Asynchronous Invocation APIs	12/15/2021

TABLE OF CONTENTS

REVISION HISTORY	i
TABLE OF CONTENTS	ii
1. 概述	1
1.1. 模块说明	1
1.2. 关键字说明	1
2. API 参考	3
2.1. MI_DSP_CreateDev	3
2.2. MI_DSP_DestroyDev	4
2.3. MI_DSP_Invoke	4
2.4. MI_DSP_InvokeAsync	6
2.5. MI_DSP_WaitDone	8
3. 数据类型	10
3.1. MI_DSP_DEV_e	10
3.2. MI_DSP_DevAttr_t	11
3.3. CustReadFunc	11
3.4. MI_DSP_BufVector_t	12
3.5. MI_DSP_BufInfo_t	12
3.6. MI_DSP_PARAM_t	13
3.7. MI_DSP_PRI_e	13
3.8. MI_DSP_FW_PACKING_TYPE_e	14
4. 错误码	16
5. 示例代码	17
6. PROCFS 介绍	20
6.1. 使用 GDB 调试 firmware	20
6.1.1 概述	20
6.1.2 JTAG 接口切换	20
6.1.3 xt-ocd 工具安装	20
6.1.4 xt-ocd 工具运行	20
6.1.5 xt-ocd 配置文件	21
6.1.6 GDB 调试	22
6.2. 使用 xplorer IDE 调试 firmware	24
6.2.1 概述	24
6.2.2 IDE 调试	24

1. 概述

1.1. 模块说明

MI DSP 是 SStar 平台中内嵌的 DSP core，用于加速智能视觉开发，其中 MI 提供了 API 方便用户快速开发和深度定制，提高产品的竞争力。

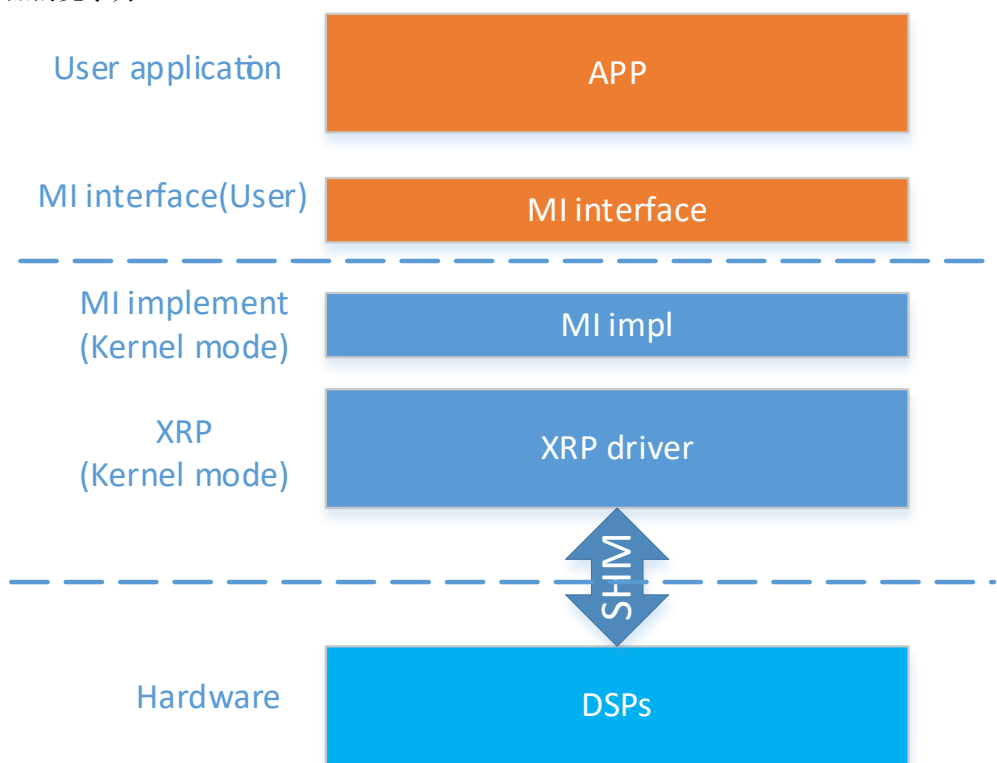


图 1-1 MI DSP 开发框架图

如上图 1-1 所示，MI DSP interface 主要提供 user mode application 的 API 接口。从 User mode 的 Application 开始，通过调用 MI DSP interface,到 kernel mode 的 MI impl 层，再到 kernel 的 driver, 最后通过 share memory 跟 DSP 交互。

1.2. 关键字说明

ID: Identity document，表示唯一编码。

MI: SStar SDK Middle Interface。

Hex: 十六进制。

Kernel Mode: 工作在 Kernel 环境下的代码，代码具有直接操作硬件的权限，比如 ko 中的函数和线程等。

User Mode: 工作在 User 环境下的代码，比如用户的应用程序，系统调用等。

XRP: Remote Processing，基于 Linux 操作系统的通信接口，用于与 DSP 交互。

APP: Application，主要指调用 MI API 的应用程序。

API: Application Programming Interface，指 Sstar SDK 提供给应用程序调用的接口。

IMPL: Implement，MI 在 kernel 层的实现。

SHM: Share memory，共享内存。

IPC: IP Camera，网络摄像机。

HW: Hardware，硬件。

2. API 参考

提供给 user application 的 MI DSP 接口如下。

API 名	功能
系统功能类	
MI_DSP_CreateDev	创建 DSP device
MI_DSP_DestroyDev	销毁 DSP device
MI_DSP_Invoke	执行一次 DSP 定制的算法，并得到输出

2.1. MI_DSP_CreateDev

➤ 功能

创建 DSP 设备。

➤ 语法

```
MI_S32 MI_DSP_CreateDev(MI\_DSP\_DEV\_e eDspDev, MI\_DSP\_DevAttr\_t *pstDspDevAttr,
CustReadFunc pReadFunc, char *pCtx, MI_U32 fwSize);
```

➤ 形参

参数名称	描述	输入/输出
eDspDev	Device ID, 对应 DSP core ID	输入
pstDspDevAttr	Device 设备属性	输入
pReadFunc	用户自定义读取文件的函数指针	输入
pCtx	Firmware 文件路径	输入
fwSize	Firmware 文件大小	输入

➤ 返回值

- 0 成功。
- 非 0 失败，参照[错误码](#)。

➤ 依赖

- 头文件：mi_dsp.h
- 库文件：libmi_dsp.a / libmi_dsp.so

※ 注意

- 当用户使用的自定义读取文件函数 pReadFunc，fwSize 需与 pCtx 路径所指定的实际文件大小相同；
- 当 fwSize 为 0 时，则 pReadFunc 需设置为 NULL；若用户使用自定义读取文件函数 pReadFunc，此为错误用法，请遵循上面的注意事项。

➤ 举例

```
MI_S32 s32Ret;
MI_DSP_DevAttr_t stDevAttr;

stDevAttr.eFwPackingType = E_MI_DSP_FW_PACKING_PLAIN;
char *pReadCtx = "/mnt/dsp_firmware";

s32Ret = MI_DSP_CreateDev(MI_DSP_DEV_0, &stDevAttr, NULL, pReadCtx, 0);
```

```
if(s32Ret != MI_SUCCESS){
    printf("Fail to create DSP device \n");
    return s32Ret;
}
```

2.2. MI_DSP_DestroyDev

- 功能
销毁 DSP 设备。
- 语法
MI_S32 MI_DSP_DestroyDev([MI_DSP_DEV_e](#) eDspDev);

- 形参

参数名称	描述	输入/输出
eDspDev	Device ID, 对应 DSP core ID	输入

- 返回值
 - 0 成功。
 - 非 0 失败，参照[错误码](#)。
- 依赖
 - 头文件：mi_dsp.h
 - 库文件：libmi_dsp.a / libmi_dsp.so
- ※ 注意
 - u32Dev 必须是之前创建的 DSP device。

2.3. MI_DSP_Invoke

- 功能
执行一次 DSP 的算法。
- 语法
MI_S32 MI_DSP_Invoke([MI_DSP_DEV_e](#) eDspDev, [MI_DSP_BufVector_t](#) *pstInputBuf,
MI_DSP_BufVector_t *pstOutputBuf, [MI_DSP_PARAM_t](#) *pstParam, [MI_DSP_PRI_e](#) ePri , MI_U16
u16TimeOutMs);

- 形参

参数名称	描述	输入/输出
eDspDev	Device ID, 对应 DSP core ID	输入
pstInputBuf	输入 DSP 的数组结构体指针	输入
pstOutputBuf	DSP 输出的数组结构体指针	输入

pstParam	输入 DSP 的数组结构体指针	输入/输出
ePri	此次 Invoke 的优先级	输入
u16TimeOutMs	此次 Invoke 的超时时间	输入

➤ 返回值

- 0 成功。
- 非 0 失败，参照[错误码](#)。

➤ 依赖

- 头文件：mi_dsp.h
- 库文件：libmi_dsp.a / libmi_dsp.so

※ 注意

- [MI_DSP_Invoke](#) 是阻塞式 API，执行一次得到一次结果。
- pstParam->s32Result 返回该次 invoke DSP 端的处理结果指示。

➤ 举例

```
MI_DSP_BufVector_t stInputBuf, stOutputBuf;
MI_DSP_PARAM_t stParam;
MI_S32 retVal = 0;

memset(&stInputBuf, 0, sizeof(MI_DSP_BufVector_t));
memset(&stOutputBuf, 0, sizeof(MI_DSP_BufVector_t));

/* 1、通过 MMA 分配 InputBuf 的空间，并对其进行映射 */
stInputBuf.astBufInfoVec[0].u32Size = 0x1000;
MI_SYS_MMA_Alloc(0, NULL, stInputBuf.astBufInfoVec[0].u32Size, &stInputBuf.astBufInfoVec[0].phyAddr);
MI_SYS_Mmap(stInputBuf.astBufInfoVec[0].phyAddr, stInputBuf.astBufInfoVec[0].u32Size, &virtAddrIn,
FALSE);
stInputBuf.u32BufCount = 1;
stInputBuf.astBufInfoVec[0].u32Flag = 0x1;
memset(virtAddrIn, 0, stInputBuf.astBufInfoVec[0].u32Size);

/* 2、通过 MMA 分配 OutputBuf 的空间，并对其进行映射 */
stOutputBuf.astBufInfoVec[0].u32Size = 0x1000;
MI_SYS_MMA_Alloc(0, NULL, stOutputBuf.astBufInfoVec[0].u32Size, &stOutputBuf.astBufInfoVec[0].phyAddr);
MI_SYS_Mmap(stOutputBuf.astBufInfoVec[0].phyAddr, stOutputBuf.astBufInfoVec[0].u32Size, &virtAddrOut,
FALSE);
stOutputBuf.u32BufCount = 1;
stOutputBuf.astBufInfoVec[0].u32Flag = 0x2;
stParam.u64Data[0] = 0x2222;
memset(virtAddrOut, 0, stOutputBuf.astBufInfoVec[0].u32Size);

/* 3、对 InputBuf 进行赋值 */
for(int i = 0; i < stInputBuf.astBufInfoVec[0].u32Size; i++) {
    char* p = (char*)virtAddrIn;
    p[i] = 0xc2;
```



```

}

/* 4、调用一次 dsp core0 的算法 */
retVal = MI_DSP_Invoke(0, &stInputBuf, &stOutputBuf, &stParam, 0, 65535);
if (retVal != MI_SUCCESS)
{
    printf("core 0 invoke err, retVal: 0x%x!\n", retVal);
    goto invoke_err;
}

/* 5、打印处理结果 */
for(int i = 0; i < 10; i++) {
    char* p = (char*)virtAddrOut;
    printf("%d \n", p[i]);
}

```

2.4. MI_DSP_InvokeAsync

➤ 功能

提交一次异步 DSP 运算任务。

➤ 语法

```
MI_S32 MI_DSP_InvokeAsync(MI_DSP_DEV_e eDspDev, MI_DSP_BufVector_t *pstInputBuf,
MI_DSP_BufVector_t *pstOutputBuf, MI_DSP_PARAM_t *pstParam, MI_DSP_PRI_e ePri, MI_U32
*u32fence);
```

➤ 形参

参数名称	描述	输入/输出
eDspDev	Device ID, 对应 DSP core ID	输入
pstInputBuf	输入 DSP 的数组结构体指针	输入
pstOutputBuf	DSP 输出的数组结构体指针	输入
pstParam	输入 DSP 的数组结构体指针	输入/输出
ePri	此次 Invoke 的优先级	输入
u32fence	此次 Invoke 的任务标识序号	输出

➤ 返回值

- 0 成功。
- 非 0 失败，参照[错误码](#)。

➤ 依赖

- 头文件：mi_dsp.h
- 库文件：libmi_dsp.a / libmi_dsp.so

※ 注意

- [MI_DSP_InvokeAsync](#)是非阻塞式 API，执行一次立即返回，如果返回 MI_DSP_ERR_PRIQ_FULL 则意味着对应 DSP 设备处理比较繁忙，任务队列已经满了。需要配合 [MI_DSP_WaitDone](#) 函数等待任务队列有空位，再提交计算任务。

- 参数 `u32fence` 是函数调用返回的标志此次计算任务的 ID，使用 [MI_DSP_WaitDone](#) 函数等待这次计算任务完成。
- 和 [MI_DSP_Invoke](#) 不同，`pstParam->s32Result` 不会被赋值，需要用 [MI_DSP_WaitDone](#) 函数取得结果。

➤ 举例

InputBuf 和 OutputBuf 使用 MI_SYS 模块已经分配的 buffer

```
MI_U32 lastfence = 0; int tmp = 5;
while (tmp)
{
    for (int i = 0; i < stInputBuf.astBufInfoVec[0].u32Size; i++)
    {
        char* p = (char*)virtAddrIn;
        p[i] = tmp;
    }

    MI_U32 fence = 0;
    while(MI_SUCCESS != MI_DSP_InvokeAsync(0, &stInputBuf, &stOutputBuf, &stParam, 0, &fence))
    {
        printf("dev busy %d \n",tmp);
        //usleep(19);
        if(MI_DSP_WaitDone(0,0, lastfence, NULL, 20000)!=MI_SUCCESS)
        {
            printf("dev busy %d timeout\n",tmp);
        }
    }
    lastfence = fence;
    //MI_DSP_Invoke(0, &stInputBuf, &stOutputBuf, &stParam, 0, 100);
    // if(tmp == 3)
    //     MI_DSP_ResetDev(0);
    for (int i = 0; i < 10; i++)
    {
        char* p = (char*)virtAddrOut;
        printf("%d \n", p[i]);
    }
    printf("fence:%d\n", fence);
    tmp--;
}
```

2.5. MI_DSP_WaitDone

➤ 功能

等待一次异步 DSP([MI_DSP_InvokeAsync](#))计算任务完成。

➤ 语法

`MI_S32 MI_DSP_WaitDone(MI_DSP_DEV_e eDspDev, MI_DSP_PRI_e ePri, MI_U32 u32fence, MI_S32* s32Result, MI_U32 u32TimeOutMs);`

➤ 形参

参数名称	描述	输入/输出
eDspDev	Device ID, 对应 DSP core ID	输入
ePri	对应 Async Invoke 的优先级	输入
u32fence	对应 Async Invoke 的任务 ID	输入
s32Result	返回最近的完成的任务的结果	输出
u32TimeOutMs	等待 Async Invoke 任务完成的超时时间	输入

➤ 返回值

- 0 成功。
- 非 0 失败，参照[错误码](#)。

➤ 依赖

- 头文件：mi_dsp.h
- 库文件：libmi_dsp.a / libmi_dsp.so

※ 注意

- 本函数是以 DSP 设备为单位的阻塞式调用
- 函数返回值为：MI_SUCCESS 或者 MI_DSP_ERR_TIMEOUT
- 本函数接受对应设备优先级[eDspDev][ePri]的 u32Fence 参数 ID，对 ID 的先后顺序并无要求。如果先等待后面 invoke 对应的任务 ID 完成，就意味着前面的任务也已经完成，此时调用函数等待之前的任务 ID 将会立即返回成功。
- s32Result 的值是 DSP 最新完成的 invoke 任务的值，不一定是对应 u32fence 任务的结果。考虑到 u32fence 可能是非常老旧的任务 ID 了。

➤ 举例

InputBuf 和 OutputBuf 使用 MI_SYS 模块已经分配的 buffer

```
MI_U32 lastfence = 0; int tmp = 5;
while (tmp)
{
    for (int i = 0; i < stInputBuf.astBufInfoVec[0].u32Size; i++)
    {
        char* p = (char*)virtAddrIn;
        p[i] = tmp;
    }

    MI_U32 fence = 0;
    while(MI_SUCCESS != MI_DSP_InvokeAsync(0, &stInputBuf, &stOutputBuf, &stParam, 0, &fence))
    {
        printf("dev busy %d \n",tmp);
        //usleep(19);
        if(MI_DSP_WaitDone(0,0, lastfence, NULL, 20000)!=MI_SUCCESS)
        {
            printf("dev busy %d timeout\n",tmp);
        }
    }
}
```

```

    lastfence = fence;
    //MI_DSP_Invoke(0, &stInputBuf, &stOutputBuf, &stParam, 0, 100);
    // if(tmp == 3)
    //     MI_DSP_ResetDev(0);
    for (int i = 0; i < 10; i++)
    {
        char* p = (char*)virtAddrOut;
        printf("%d \n", p[i]);
    }
    printf("fence:%d\n", fence);
    tmp--;
}

```

3. 数据类型

相关数据类型、数据结构定义如下。

数据类型	定义
MI_DSP_DEV_e	定义设备 Device 的类型
MI_DSP_DevAttr_t	定义设备的硬件属性
CustReadFunc	用于客制化的读取文件方式，譬如加密等
MI_DSP_BufVector_t	定义多个输入/输出 Buffer 的结构体
MI_DSP_BufInfo_t	定义码流信息的 Buffer 结构体
MI_DSP_PARAM_t	定义客制化的用户参数，立即数或地址/长度
MI_DSP_PRI_e	Invoke API 的优先级
MI_DSP_FW_PACKING_TYPE_e	定义 DSP firmware 的打包方式

3.1. MI_DSP_DEV_e

➤ 说明

定义设备 Device 的类型。

➤ 定义

```

typedef enum
{
    MI_DSP_DEV_0 = 0,
    MI_DSP_DEV_1,
    MI_DSP_DEV_2,
    MI_DSP_DEV_3,
    MI_DSP_DEV_BUTT
} MI_DSP_DEV_e;

```

➤ 成员

成员名称	描述
MI_DSP_DEV_0	DSP device 设备 Core 0
MI_DSP_DEV_1	DSP device 设备 Core 1
MI_DSP_DEV_2	DSP device 设备 Core 2
MI_DSP_DEV_3	DSP device 设备 Core 3

※ 注意事项
N/A

➤ 相关数据类型及接口
N/A

3.2. MI_DSP_DevAttr_t

➤ 说明
定义设备 Device 的属性。

➤ 定义

```
typedef struct MI_DSP_DevAttr_s
{
    MI_DSP_FW_PACKING_TYPE_e eFwPackingType;
    MI_U32                    u32ResetVectorAddr;
    MI_U16                    u16Prid;
    MI_BOOL                   bRebootFw;
} MI_DSP_DevAttr_t;
```

➤ 成员

成员名称	描述
eFwPackingType	Firmware 的打包类型
u32ResetVectorAddr	Firmware 的 reset vector 地址
u16Prid	DSP 设备指定的 processor id, debug 时需设置为 0x272
bRebootFw	Invoke 超时是否自动 reset firmware

※ 注意事项

- 当在 IDE create Custom LSPs 时选定 External 选项用以自定义 address, u32ResetVecAddr 需与该 address 一致, 且该值要 64 对齐; 当选定 Primary 选项, u32ResetVecAddr 需设置为 0。
- 当 firmware 中部分函数存放在 iram, 则 eFwPackingType 为 E_MI_DSP_FW_PACKING_ROMING, 故需要修改编译 firmware 的 LSP——memmap.xmm 文件添加 ROMING=true 与 specs 文件使用 -lhandler-reset-unpack 替换原来的 -lhandler-reset, 具体查看 LSP Manual Chapter 3 Placing Applications in ROM; 若没有函数存放在 iram, 则 eFwPackingType 设置为 E_MI_DSP_FW_PACKING_PLAIN。

➤ 相关数据类型及接口
N/A

3.3. CustReadFunc

➤ 说明

用于定制化的读取文件方式，需注意该函数执行成功时返回值应大于等于 0。

➤ 定义

```
typedef int (* CustReadFunc)(void *dst_buf, int offset, int size, char *ctx)
```

➤ 成员

成员名称	描述
dst_buf	数据存放地址
offset	文件指针偏移量
size	读取文件大小
ctx	文件路径

※ 注意事项

- 当用户需要自定义读取文件的时候(如加密)，可以设置此函数指针。
- 该函数所指定的 dst 空间，是由 MI_DSP_CreateDev()函数内部进行分配的，大小为 fwSize，用户不需分配。
- 当用户将该函数设置为 NULL 时，使用默认的读取 firmware 函数，默认读取函数不需用户提供。

➤ 相关数据类型及接口

N/A

3.4. MI_DSP_BufVector_t

➤ 说明

定义多个输入/输出 Buffer 的结构体。

➤ 定义

```
typedef struct MI_DSP_BufVector_s
{
    MI_U32 u32BufCount;
    MI_DSP_BufInfo_t astBufInfoVec[MAX_BUF_CNT];
} MI_DSP_BufVector_t;
```

➤ 成员

成员名称	描述
u32BufCount	BufInfo 的个数
astBufInfoVec	BufInfo 的数组

※ 注意事项

- MAX_BUF_CNT 推荐值是 10，该宏定义在 mi_dsp_datatype.h 中指定：#define MAX_BUF_CNT 10。

➤ 相关数据类型及接口

N/A

3.5. MI_DSP_BufInfo_t

➤ 说明

定义码流信息的结构体。

➤ 定义

```
typedef struct MI_DSP_BufInfo_s
{
    MI_PHY phyAddr;
    MI_U32 u32Size;
    MI_U32 u32Flag;
} MI_DSP_BufInfo_t;
```

➤ 成员

成员名称	描述
phyAddr	传给 DSP 的内存的物理地址
u32Size	传给 DSP 的内存的大小
u32Flag	传给 DSP 的内存的一些标志信息

※ 注意事项

- 通过 MI_SYS 的接口获取连续的物理内存。

➤ 相关数据类型及接口

N/A

3.6. MI_DSP_PARAM_t

➤ 说明

定义定制化的用户参数，可以是立即数或地址/长度。

➤ 定义

```
typedef struct MI_DSP_PARAM_s
{
    MI_U64 u64Data[MAX_PARA_CNT];
    MI_S32 s32Result;
} MI_DSP_PARAM_t;
```

➤ 成员

成员名称	描述
u64Data	自定义数组，用于向 DSP 的 message handler 发送 invoke 参数
s32Result	MI_DSP_Invoke 调用时，存放 DSP 的 message handler 的返回结果

※ 注意事项

- MAX_PARA_CNT 推荐值是 2，该宏定义在 mi_dsp_datatype.h 中指定：#define MAX_PARA_CNT 2。

- 在 [MI_DSP_Invoke](#) 时，数组可存放定制化的用户参数，若存放的是地址，必须是 MI_SYS_MMA_ALLOC 申请的 buffer 的物理地址，便于 DSP 查看。
- s32Result 仅在 [MI_DSP_Invoke](#) 调用时才会被赋值。

- 相关数据类型及接口
N/A

3.7. MI_DSP_PRI_e

- 说明
定义 [MI_DSP_Invoke](#) API 的优先级，表示 dsp 所处理 msg 的队列的优先级。

- 定义
typedef enum
{
 MI_DSP_PRI_0 = 0,
 MI_DSP_PRI_1,
 MI_DSP_PRI_2,
 MI_DSP_PRI_BUTT
} MI_DSP_PRI_e;

- 成员

成员名称	描述
MI_DSP_PRI_0	优先队列级别 0，最高
MI_DSP_PRI_1	优先队列级别 1
MI_DSP_PRI_2	优先队列级别 2，最低

- ※ 注意事项
N/A

- 相关数据类型及接口
N/A

3.8. MI_DSP_FW_PACKING_TYPE_e

- 说明
定义 firmware 的打包类型。

- 定义
typedef enum
{
 E_MI_DSP_FW_PACKING_ROMING = 0,
 E_MI_DSP_FW_PACKING_PLAIN,
 E_MI_DSP_FW_PACKING_BUTT
} MI_DSP_FW_PACKING_TYPE_e;

➤ 成员

成员名称	描述
E_MI_DSP_FW_PACKING_ROMING	Firmware 打包为 ROMING 类型
E_MI_DSP_FW_PACKING_PLAIN	Firmware 打包为 PLAIN 类型

※ 注意事项

- 当 firmware 中部分函数存放在 iram，则 eFwPackingType 为 E_MI_DSP_FW_PACKING_ROMING，故需要修改编译 firmware 的 LSP——memmap.xmm 文件添加 ROMING=true 与 specs 文件使用 -lhandler-reset-unpack 替换原来的 -lhandler-reset，具体查看 LSP Manual Chapter 3 Placing Applications in ROM；若没有函数存放在 iram，则 eFwPackingType 设置为 E_MI_DSP_FW_PACKING_PLAIN。

➤ 相关数据类型及接口

N/A

4. 错误码

MI DSP 模块 错误码如下表所示。

错误码	宏定义	描述
0xA0242080	MI_DSP_ERR_READ_FILE	读取文件错误
0xA0242081	MI_DSP_ERR_INVALID_DEVID	设备号错误或超出硬件支持范围
0xA0242082	MI_DSP_ERR_DEV_ATTR	Device 设备属性参数错误
0xA0242083	MI_DSP_ERR_RE_CREATE	重复创建已存在的设备
0xA0242084	MI_DSP_ERR_NULL_PTR	函数参数中有不允许为空的指针
0xA0242085	MI_DSP_ERR_PRIQ_FULL	Priority Queue 满
0xA0242086	MI_DSP_ERR_NOT_READY	系统没有初始化或 Firmware 没有 ready
0xA0242087	MI_DSP_ERR_TIMEOUT	系统异常导致超时
0xA0242088	MI_DSP_ERR_INVALID_PARAM	参数超出合法范围
0xA0242089	MI_DSP_ERR_BAD_ADDRESS	地址非法
0xA024208A	MI_DSP_ERR_FAILED	其他错误

5. 示例代码

```
#include <stdio.h>
#include <unistd.h>
#include <stdlib.h>
#include <string.h>

#include <mi_dsp.h>
#include <mi_dsp_datatype.h>
#include <mi_sys.h>
```

```

int main()
{
    MI_S32 retVal = 0;
    MI_DSP_DevAttr_t stDspDevAttr;
    MI_DSP_BufVector_t stInputBuf, stOutputBuf;
    MI_DSP_PARAM_t stParam;

    void* virtAddrIn = 0;
    void* virtAddrOut = 0;
    stDspDevAttr.eFwPackingType = E_MI_DSP_FW_PACKING_PLAIN;
    stDspDevAttr.u32ResetVecAddr = 0x10040000;
    stDspDevAttr.u16Prid = 0x272;

    /* 1、创建 dsp core0 的设备，并指定 firmware 路径 */
    retVal = MI_DSP_CreateDev(0, &stDspDevAttr, NULL,
        "/lib/firmware/tc_invoke.elf", 0);
    if (retVal != MI_SUCCESS)
    {
        printf("core 0 createdev err, retVal: 0x%x!\n", retVal);
        goto create_err;
    }

    memset(&stInputBuf, 0, sizeof(MI_DSP_BufVector_t));
    memset(&stOutputBuf, 0, sizeof(MI_DSP_BufVector_t));

    /* 2、通过 MMA 分配 InputBuf 的空间，并对其进行映射 */
    stInputBuf.astBufInfoVec[0].u32Size = 0x1000;
    MI_SYS_MMA_Alloc(0, NULL, stInputBuf.astBufInfoVec[0].u32Size,
        &stInputBuf.astBufInfoVec[0].phyAddr);
    MI_SYS_Mmap(stInputBuf.astBufInfoVec[0].phyAddr,
        stInputBuf.astBufInfoVec[0].u32Size, &virtAddrIn, FALSE);
    stInputBuf.u32BufCount = 1;
    stInputBuf.astBufInfoVec[0].u32Flag = 0x1;
    memset(virtAddrIn, 0, stInputBuf.astBufInfoVec[0].u32Size);

    /* 3、通过 MMA 分配 OutputBuf 的空间，并对其进行映射 */
    stOutputBuf.astBufInfoVec[0].u32Size = 0x1000;
    MI_SYS_MMA_Alloc(0, NULL, stOutputBuf.astBufInfoVec[0].u32Size,
        &stOutputBuf.astBufInfoVec[0].phyAddr);
    MI_SYS_Mmap(stOutputBuf.astBufInfoVec[0].phyAddr,
        stOutputBuf.astBufInfoVec[0].u32Size, &virtAddrOut, FALSE);
    stOutputBuf.u32BufCount = 1;
    stOutputBuf.astBufInfoVec[0].u32Flag = 0x2;
    stParam.u64Data[0] = 0x2222;

```

```

memset(virtAddrOut, 0, stOutputBuf.astBufInfoVec[0].u32Size);

/* 4、对 InputBuf 进行赋值 */
for(int i = 0; i < stInputBuf.astBufInfoVec[0].u32Size; i++) {
    char* p = (char*)virtAddrIn;
    p[i] = 0xc2;
}

/* 5、调用一次 dsp core0 的算法 */
retVal = MI_DSP_Invoke(0, &stInputBuf, &stOutputBuf, &stParam, 0,
65535);
if (retVal != MI_SUCCESS)
{
    printf("core 0 invoke err, retVal: 0x%x!\n", retVal);
    goto invoke_err;
}

/* 6、打印处理结果 */
for(int i = 0; i < 10; i++) {
    char* p = (char*)virtAddrOut;
    printf("%d \n", p[i]);
}

/* 7、进行资源释放工作 */
invoke_err:
MI_SYS_Munmap(virtAddr, 0x1000);
MI_SYS_MMA_Free(0, u64PhyAddr);
MI_DSP_DestroyDev(0);

create_err:
while(1) {
    sleep(1);
}

return 0;
}

```

6. PROCFS 介绍

6.1. 使用 GDB 调试 firmware

6.1.1 概述

正确配置软件系统，在板端运行 MI_DSP APP。

在 debug 主机配置 Jlink 环境，并正确连接 JTAG usb debug 工具。

6.1.2 JTAG 接口切换

在默认板端的 SOC 线路中，JTAG 连接的是 ARM cores。需要在板端的 console 键入如下命令，将 JTAG 连接至 DSP cores。

- /customer/riu_w 101e 7b 101; /customer/riu_w 101e 7b 001; 将 JTAG 切到 DSP cores 上 后续 xt-ocd 才能连上。
- /customer/riu_w 101e 7b 0 将 JTAG 切回到 ARM cores。

6.1.3 xt-ocd 工具安装

通过 xplorer XPG 下载 xt-ocd-14.05-windows64-installer.exe，如果 debug 主机是在 Linux 环境，请下载 Linux 版本并安装。

运行 xt-ocd 需要 jlink 的库：Jlink_x64.dll(windows)，建议复制到 xt-ocd 安装目录的 modules 文件夹中。

6.1.4 xt-ocd 工具运行

xt-ocd 的运行详情请参考 cadence 官方文档：Xtensa® Debug Guide。

Xt-ocd 运行需要在 MI_DSP createdev 之后进行，在那之前 DSP cores 还没有被初始化。

基本的运行方式：在 debug 主机运行 xt-ocd.exe -c mytopology.xml。其中 mytopology.xml 为配置文件。

成功运行之后如有如下类似 log，即能够正确获取 JTAG 的 IR bits 跟 width。

```
D:\Program Files (x86)\Tensilica\Xtensa OCD Daemon 14.05>xt-ocd.exe -c mytopology.xml
XOCD 14.05 2020-10-12 17:38:02
(c) 1999-2021 Cadence Design Systems Inc. All rights reserved.
[Debug Log 2021-03-04 16:54:53]
Loading module "gdbstub" v2.0.0.12
Loading module "jlink" v2.0.2.0
Using JLINK lib v.61210
Jlink USB Serial Number: 261003840
Connected to Jlink Device:
  Name:'J-Link V11 compiled Aug 14 2019 16:21:09'
  S/N:261003840
  Firmware: J-Link V11 compiled Aug 14 2019 16:21:09
  Requested/Set TCK: 2000kHz/2000kHz
Loading module "jtag" v2.0.0.20
Loading module "xtensa" v2.0.0.48
Loading module "traxapp" v2.0.0.8
Loading module "trax" v2.0.1.23
Starting thread 'GDBStub'
Starting thread 'TraxApp'
Opened GDB socket at port 20000
traxapp: listening on port 11444
Initialize XDM driver
Total IR bits : 5
TAP[0] irwidth = 5
```

6.1.5 xt-ocd 配置文件

```
<configuration>
  <controller id='Controller0' module='jlink' usbser='261008694' speed='10000000' />
  <!--
    Default Flyswatter2
  -->
  <controller id='Controller0' module='ft2232' probe='flyswatter2' speed='10MHz' />

  Here are other example controller (probe) lines, that can replace the above one
  when using different JTAG probes (scan controllers):

  Cadence Design Systems ML605 Daughterboard - optional usbser is inventory sticker number prefixed with 'ML605-':
  <controller id='Controller0' module='ft2232' probe='ML605' speed='10MHz' usbser='ML605-2147' />

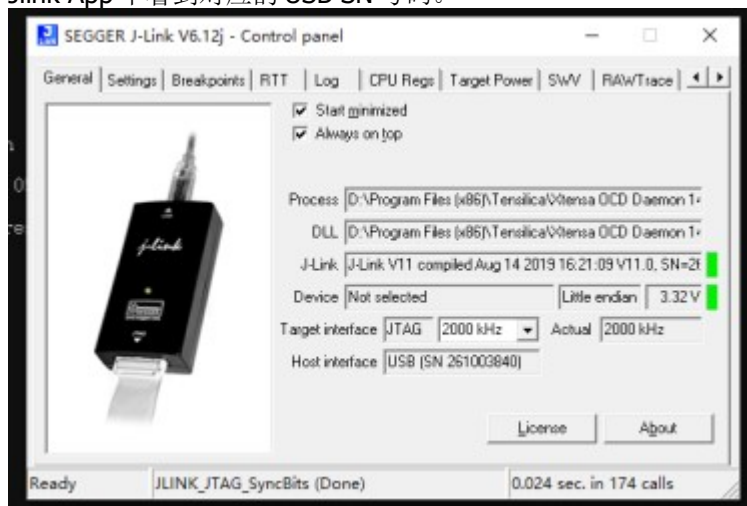
  Tin Can Tools Flyswatter2:
  <controller id='Controller0' module='ft2232' probe='flyswatter2' speed='10MHz' />

  RVI/DSTREAM: NOTE: Probe settings are in the loaded rvc file
  <controller id='Controller0' module='rvi' />

  JLink IP (10MHz JTCK):
  <controller id='Controller0' module='jlink' ipaddr='192.168.1.1' port='0' speed='10000000' />

  JLink USB (10MHz JTCK):
  <controller id='Controller0' module='jlink' usbser='12345678' speed='10000000' />
  -->
  <driver id='XtensaDriver0' module='xtensa' step-intr='mask,stepover,setps' />
  <driver id='TraxDriver0' module='trax' />
  <chain controller='Controller0'>
    <tap id='TAP0' irwidth='5' />
  </chain>
  <system module='jtag'>
    <component id='Component0' tap='TAP0' config='trax' />
  </system>
  <device id='Xtensa0' component='Component0' driver='XtensaDriver0' />
  <device id='Trax0' component='Component0' driver='TraxDriver0' />
  <application id='GDBStub' module='gdbstub' port='20000'>
    <target device='Xtensa0' />
  </application>
  <application id='TraxApp' module='traxapp' port='11444'>
    <target device='Trax0' />
  </application>
</configuration>
```

配置文件中 **usbser='261008694'**，项目需要根据实际情况填写 jlink 的 usb S/N 号码。可以在 DEBUG 主机的 Jlink App 中看到对应的 USB SN 号码。



配置文件中的速度可以适当调快。在 FPGA 环境中，如果的程序比较大/交互信息多，过快会导致沟通失败。在 **xt-ocd** 的运行窗口可看到失败异常信息。

6.1.6 GDB 调试

1. 在本地调试 console 运行 `xt-gdb: xt-gdb.exe --xtensa-core=T12_DSP_eva HelloWorld1`
2. 连接 HAPS 远程环境 `target remote [host]:[port]: target remote 172.19.32.104:20000`
3. 关闭自动本地连接: `set auto-connect-native-target off`
4. load 程序: `load`
5. continue 运行程序: `continue`

+ 附上一份运行 log:

```

E:\usr\xtensa\XtDevTools\install\tools\RI-2020.5-win32\XtensaTools\bin>xt-gdb.exe --xtensa-
core=T12_DSP_fpga HelloWorld1
GNU gdb (GDB) 7.11.1 Xtensa Tools 14.05
Copyright (C) 2016 Free Software Foundation, Inc.
License GPLv3+: GNU GPL version 3 or later <http://gnu.org/licenses/gpl.html>
This is free software: you are free to change and redistribute it.
There is NO WARRANTY, to the extent permitted by law. Type "show copying"
and "show warranty" for details.
This GDB was configured as "--host=x86_64-w64-mingw32 --target=xtensa-elf".
Type "show configuration" for configuration details.
For bug reporting instructions, please see:
<http://www.gnu.org/software/gdb/bugs/>.
Find the GDB manual and other documentation resources online at:
<http://www.gnu.org/software/gdb/documentation/>.
For help, type "help".
Type "apropos word" to search for commands related to "word"...
Reading symbols from HelloWorld1...done.
(xt-gdb) target remote 172.19.32.104:20000

```

```
Remote debugging using 172.19.32.104:20000
0x00000000 in ?? ()
(xt-gdb) set auto-connect-native-target off
(xt-gdb) load
Loading section .DispatchVector.text, size 0x25c lma 0x10000000
Loading section .DispatchHandler.text, size 0x4 lma 0x1000025c
Loading section .ResetVector.text, size 0x4 lma 0x10004000
Loading section .ResetHandler.text, size 0x18c lma 0x10004004
Loading section .clib.rodata, size 0x618 lma 0x20000000
Loading section .rtos.rodata, size 0x80 lma 0x20000640
Loading section .rodata, size 0x218 lma 0x200006c0
Loading section .text, size 0xa298 lma 0x200008e0
Loading section .clib.data, size 0x1bc lma 0x2000ab80
Loading section .data, size 0x233c lma 0x2000ad40
Start address 0x10004000, load size 54320
Transfer rate: 26 KB/sec, 1597 bytes/write.
(xt-gdb) continue
Continuing.
main 34
main 36 cc
main 36 cc
main 36 cc
main 36 cc
main 36 cc
main 36 cc
main 36 cc
main 36 cc
main 36 cc
main 36 cc
main 36 cc
main 36 cc
main 36 cc
hello world 0

Program received signal SIGTRAP, Trace/breakpoint trap.
_exit () at /build/tree/RI-2020.5_kuma/ib/tools/swtools-MSWin32-x86/xtensa-elf/src/xtos/exit.S:88
88      /build/tree/RI-2020.5_kuma/ib/tools/swtools-MSWin32-x86/xtensa-elf/src/xtos/exit.S: No such file or
r directory.
(xt-gdb)
```

※ **Xt-gdb** 的调试手册请参考 **cadence** 的官方文档: **GNU Debugger User's Guide**

6.2. 使用 xplorer IDE 调试 firmware

6.2.1 概述

除了使用 **xt-gdb** 命令调试 **firmware** 之外, 还可以使用 **xplorer IDE** 。

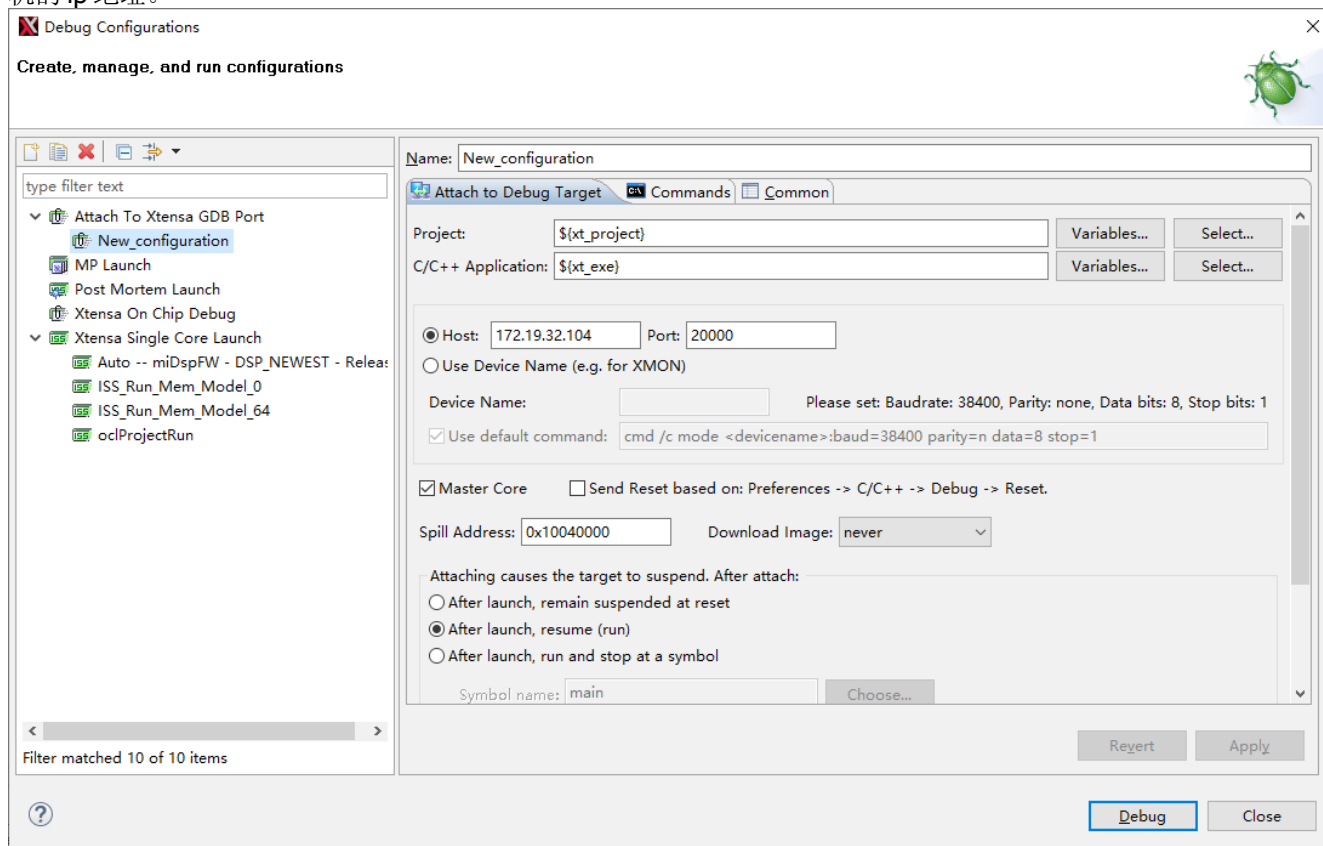
IDE 调试与 gdb 直接调试需要的准备工作一样，都需要执行 5.1.章节的 5.1.1 到 5.1.5。

6.2.2 IDE 调试

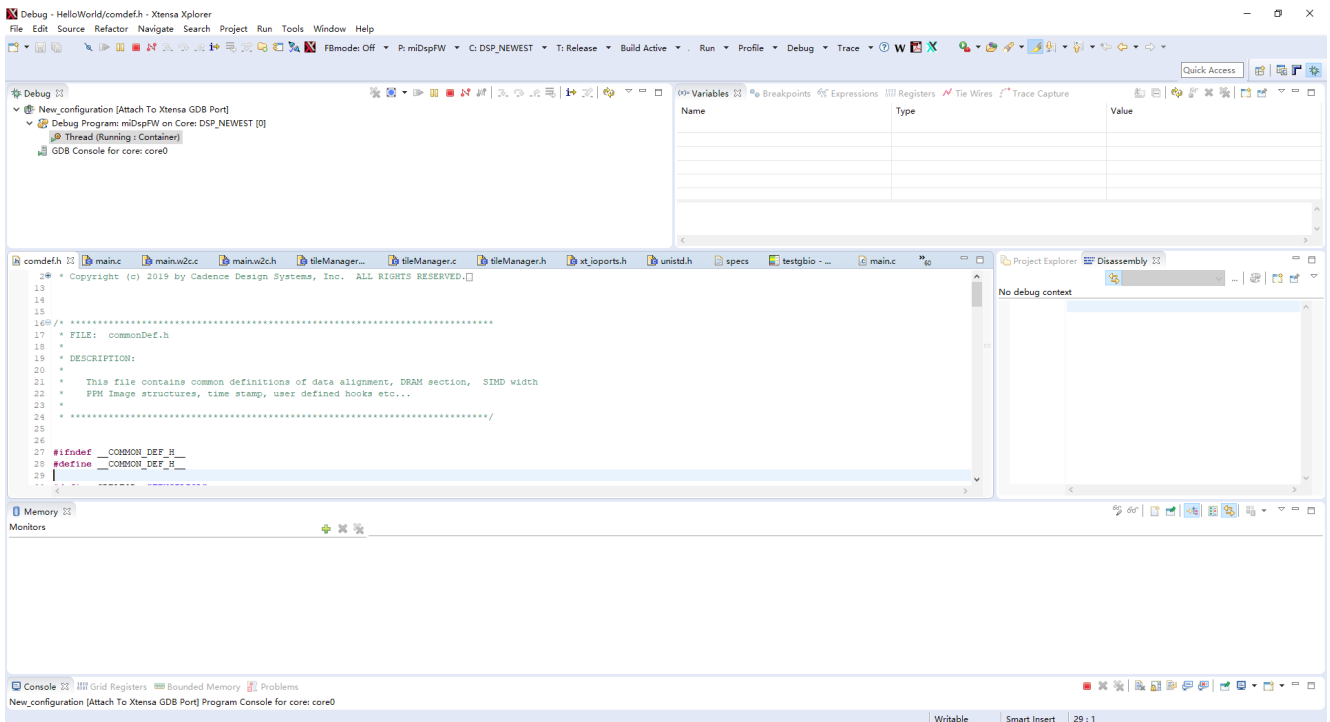
使用 xplorer 连接 dsp，在 xplorer 工具栏中配置正在运行的 firmware，在 debug 选项中选择 debug configuration n。



如下图所示，新建一个 attach to xtensa gdb port 配置，并按图选择配置项目。注意 host IP 地址请填写 haps 主机的 ip 地址。



连接上 DSP 之后，可以看到 DSP 处于 running 状态。



选择 **pause->||**，然后选择 **pause**，此时 DSP 会 **stall** 住，这个时候可以通过 IDE 查看系统的各个信息。
如：运行栈，内存信息，变量信息，寄存器信息等。

